



PROCESADORES DE LENGUAJE - trad1

Grupo 206

11/03/2025

Jorge Mejías Donoso 100495807@alumnos.uc3m.es

Alberto Menchen Montero 100495692@alumnos.uc3m.es

1. Introducción

En esta segunda sesión se ha ampliado el traductor desarrollado previamente, pasando de una conversión de expresiones aritméticas aisladas a la traducción de fragmentos más completos del lenguaje C, incluyendo declaraciones de variables, sentencias condicionales, bucles y la función principal main.

El objetivo principal ha sido generar una salida en notación prefija que sea completamente compatible con el entorno Lisp, permitiendo traducir pequeños programas en C para su evaluación en entornos funcionales. Para ello, se ha redefinido y extendido la gramática original utilizando Bison, manteniendo una estructura modular y jerárquica.

2. Descripción del Código

El archivo trad1.y ha sido modificado para incluir nuevas reglas que permiten manejar estructuras propias del lenguaje C. A continuación se resumen las principales funcionalidades incorporadas:

Declaraciones de variables:

Se permite declarar variables tipo int, tanto de forma individual como múltiple, con o sin valor inicial. Las variables declaradas se traducen a instrucciones setq, asignando 0 por defecto si no se especifica valor.

Función principal (main):

Se ha incluido una regla específica para reconocer la función main() y traducir su cuerpo como una definición de función Lisp (defun). Al final del programa, se añade automáticamente la llamada a main para su ejecución.

Sentencias de control:

Se han añadido reglas para traducir estructuras if, if-else y while, manteniendo la notación prefija y la indentación compatible con Lisp. Estas estructuras permiten evaluar condiciones y ejecutar bloques de código según el flujo del programa.

Asignaciones y expresiones:

Se mantienen y amplían las reglas para traducir expresiones aritméticas y asignaciones, combinándose correctamente con las nuevas estructuras de control.

Instrucciones de impresión:

Se conserva el uso del operador @ para imprimir expresiones, que se traduce directamente como (print ...) en la salida.

Uso del punto y coma (;):

Se refuerza la necesidad de finalizar todas las instrucciones con ;, incluso aquellas contenidas en bloques de control, para evitar errores de análisis sintáctico.

3. Casos de Prueba

Se ha creado un fichero de entrada con el siguiente contenido (resumen):

- Declaración múltiple de variables globales (**x**, **y**, **z**)
- Función **main()** con:
 - Instrucciones de impresión
 - Asignaciones
 - Sentencias **if** y **if-else**
 - Bucle **while** con decremento de variable

La salida generada por el traductor es una secuencia de instrucciones Lisp con la estructura esperada, que ha sido verificada en el entorno online de Common Lisp.

Fragmento de prueba:

Entrada:

```
int x = 3, y, z = 10;

main() {
    @ x;
    y = 0;
    if (x > 2) { y = 1; };
    if (y > 0) { z = 5; } else { z = -5; };
    while (z > 0) { @ z; z = z - 1; };
};
```

Salida esperada:

```
(setq x 3)
(setq y 0)
(setq z 10)
(defun main ()
  (print x)
  (setq y 0)
  (if (> x 2) (setq y 1))
  (if (> y 0) (setq z 5) (setq z (- 5)))
  (while (> z 0)
    (print z)
    (setq z (- z 1)))
  )
(main)
```

5. Conclusiones

Esta sesión ha supuesto un paso importante en la evolución del traductor. Se ha ampliado la gramática para poder manejar fragmentos de código más realistas, lo que permite simular estructuras comunes en C y traducirlas de forma correcta a Lisp.

Entre los avances destacables:

- Traducción de estructuras de control y funciones.
- Soporte completo para declaraciones de variables.
- Producción de código Lisp funcional y ejecutable.

Se han superado dificultades relacionadas con la sintaxis (como el uso obligatorio de ;) y se ha validado el funcionamiento general mediante pruebas en un intérprete Lisp real.